

Tabla comparativa final del taller

Taller GA — ORM vs. SQL puro | JDBC puro con PreparedStatement vs. Spring Data JPA + Hibernate

Criterio	JDBC puro con PreparedStatement	Spring Data JPA + Hibernate
(1) Líneas de código (repositorio + conexión)	≈ 65 líneas efectivas (Conexion.java: 14 + ProductoRepositorioJdbc.java, métodos listar/crear/eliminar: 51). Sin contar buscarPorNombreSeguro/Inseguro, que son solo para la demo de inyección SQL.	≈ 60 líneas efectivas (Producto.java: 54 + ProductoRepository.java: 6). AppConfig.java (47 líneas) no se cuenta porque reemplaza la configuración que en un proyecto Spring Boot normal resuelve application.yml automáticamente, no es parte del repositorio en sí.
(2) Tiempo del listado (100 filas)	34.235 ms (nanoTime) / 85.569 ms (StopWatch)	32.240 ms (nanoTime) / 7.507 ms (StopWatch)
(3) Facilidad de mantenimiento	El SQL está en cadenas Java: cambios de esquema exigen retocar cada consulta manualmente; el mapeo ResultSet→objeto es código repetitivo (boilerplate) que se repite en cada método.	El mapeo lo hace Hibernate vía anotaciones (@Entity, @Column); los métodos CRUD básicos vienen dados por JpaRepository sin escribir SQL; findByNombre se resuelve solo con el nombre del método. Menos código propio que mantener.
(4) Prevención de SQL Injection	Se previene siempre que se use PreparedStatement con marcadores ? y setXxx(). La prueba con el ataque ' OR '1'='1 confirma 0 filas devueltas en buscarPorNombreSeguro, pero el proyecto también incluye ProductoRepositorioInseguro (concatenación de cadenas) que devolvió las 100 filas, demostrando la vulnerabilidad si no se usa PreparedStatement.	Se previene por defecto: Hibernate genera siempre PreparedStatement parametrizado, visible en el log ("where p1_0.nombre=?"). La prueba con el mismo ataque ' OR '1'='1 sobre findByNombre() devolvió 0 filas, confirmando la protección automática sin código adicional.

Nota metodológica

Los tiempos reportados provienen de una única ejecución (consola.docx). El StopWatch de JPA (7.507 ms) resultó más rápido que el de JDBC (85.569 ms) en esta corrida, probablemente porque el listar() de JDBC se ejecutó como la primera consulta "en frío" tras las dos demostraciones de inyección SQL, mientras que el findAll() de JPA se benefició de la conexión ya abierta por Hibernate.